Official Report: Enhanced File Carving Script

Author: Solomon Mwaura

Date: April 27, 2026

Subject: Analysis and Improvement of a Python File Carving Utility

1. Introduction

This report details the analysis of an initial Python script designed for forensic file carving and outlines the significant improvements implemented to enhance its efficiency, robustness, and usability. The original script, while functional for basic tasks, presented several limitations that are critical in real-world digital forensic investigations, particularly when dealing with large datasets and the need for high accuracy.

2. Analysis of Original Script: Identified Issues

The initial Python script exhibited several areas requiring attention. These issues primarily impacted performance, accuracy, and flexibility, as summarized below:

Category	Issue	Impact
Performance	Reading entire image into memory	`MemoryError` for large files, slow execution
Efficiency	Naive signature search (byte-by-byte iteration)	Very slow for large files and signatures
Logic	Suboptimal footer matching logic	Incorrect carving, truncated/over-carved files, potential for overlapping recoveries
Validation	No post-carving file type validation	Recovery of invalid or corrupted files without user awareness
Flexibility	Hardcoded file paths and signatures	Limited reusability; requires manual code modification for different scenarios
Robustness	Limited error handling	Script crashes on unexpected I/O errors or other exceptions

Category	Issue	Impact
User Experience	No progress indicator for long operations	User uncertainty during lengthy processing of large images

3. Enhanced File Carving Script: Key Improvements

The revised script, named `fixed_file_carver.py`, incorporates significant enhancements to address the identified limitations, transforming it into a more robust and efficient forensic tool. The key improvements are detailed below:

3.1. Memory-Efficient Signature Scanning

The original script's major bottleneck was reading the entire disk image into memory. The improved script now employs a chunked reading approach, processing the image in manageable segments. This prevents `MemoryError` on large forensic images and significantly improves performance.

```
def find_signatures_chunked(image_path, signature, chunk_size=4096):
    # ... (implementation details)
    while current_offset < file_size:
        f.seek(current_offset)
        chunk = f.read(chunk_size + overlap_size)
        # ... (search within chunk)
```

```
            current_offset += chunk_size
            print(f"[+] Scanned {current_offset / (1024*1024):.2f} MB...",
```

3.2. Optimized Signature Search

Instead of a manual byte-by-byte comparison, the `find_signatures_chunked` function now leverages the highly optimized `bytes.find()` method. This dramatically speeds up the process of locating file headers and footers within the data stream.

3.3. Robust Footer Matching and Carving Logic

The `extract_files` function has been refined to intelligently pair headers with their corresponding footers. It now:

- Prioritizes the closest valid footer appearing after a header.
- Considers a `max_file_size` parameter to prevent over-carving.
- Uses the offset of the next header as a boundary if no suitable footer is found, ensuring that files are not excessively large or incorrectly merged.
- Supports multiple JPEG header types (e.g., `\xFF\xD8\xFF\xE0` and `\xFF\xD8\xFF\xE1`).

3.4. Post-Carving File Type Validation

A new function, `validate_jpeg`, has been introduced to perform a basic check of the carved file's magic bytes (`\xFF\xD8` for header, `\xFF\xD9` for footer). This ensures that only genuinely recovered JPEG files are retained, reducing false positives.

```
def validate_jpeg(filepath):
    try:
        with open(filepath, 'rb') as f:
```

```
            header = f.read(2)
            f.seek(-2, os.SEEK_END)
            footer = f.read(2)
            if header == b'\xFF\xD8' and footer == b'\xFF\xD9':
                return True
    except IOError:
        pass
    return False
```

3.5. Enhanced Flexibility with Command-Line Arguments

The script now utilizes the `argparse` module, allowing users to specify the input image path, output folder, chunk size, and maximum carved file size directly from the command line. This significantly improves the script's reusability and adaptability for various forensic scenarios.

```
def main():
    parser = argparse.ArgumentParser(description="Forensic file carving
    parser.add_argument("image_path", help="Path to the raw disk image
    parser.add_argument("-o", "--output_folder", default="recovered_fil
    # ... (other arguments)
    args = parser.parse_args()
```

3.6. Comprehensive Error Handling and User Feedback

More robust `try-except` blocks have been added throughout the script to gracefully handle `FileNotFoundError`, `IOError`, and other potential exceptions during file operations. Additionally, a progress indicator is now displayed during the signature scanning phase, providing valuable feedback to the user during long processing times.

4. Conclusion

The enhanced file carving script represents a significant improvement over its initial version. By addressing critical issues related to memory management, search efficiency, carving logic, and user interaction, the `fixed_file_carver.py` script is now a more reliable, efficient, and user-friendly tool for digital forensic practitioners. These improvements ensure more accurate file recovery and a smoother investigative workflow, particularly when dealing with the complexities of large-scale digital evidence.

5. Attachments

- `code_analysis.md`: Detailed analysis of the original script's issues.
- `fixed_file_carver.py`: The improved Python file carving script.